counting the four clock cycles associated with each incoming data frame. When "st_cnt" equals zero, it signifies the first clock cycle of each data frame input. During the initial clock cycle, the starter signal "st0_rdy" becomes active when "cnt0" is in an idle state (indicated as $\sim |cnt0$ in line 85). Conversely, the pulse starter signal "st1_rdy" becomes active when "cnt0" is active (expressed as $|cnt0$ in line 86) and "cnt1" is in an idle state (noted as $\sim |cnt1$ in line 86).

The design for generating counter enable signals, "cnt0_en" and "cnt1_en", is outlined in lines 88–89. These signals enable the counters when the corresponding start indicators are asserted. The counters are active until they are reset to zero. The counter designs themselves are described in lines 91–103, where they increment by decimal 3'd1 with each clock cycle when enabled. Upon reaching the maximum value of decimal 3'd5, the counters reset to decimal 3'd0.

The design for the validation signal, referred to "vld", is detailed in lines 105–112. The conclusion of data processing is signaled by the counters reaching their maximum value. Subsequently, after a single clock cycle, the valid *ddot* result is produced, and the "vld" signal is asserted, indicating the presence of valid data on the "z" output.

T&F--Not for Distribution

PBL 28: Iterative design on DDOT with streaming width one

1) Design a *ddot* circuit utilizing the signal-clock-cycle FP operators, symbolized as $z = x^T \times y$, where x and y represent FP vectors of size eight, and z signifies the resulting *ddot* value. The mathematical representation is as follows:

$$z = \begin{pmatrix} x0 & x1 & x2 & x3 & x4 & x5 & x6 & x7 \end{pmatrix} * \begin{pmatrix} y0 \\ y1 \\ y2 \\ y3 \\ y4 \\ y5 \\ y6 \\ y7 \end{pmatrix} \tag{10.2}$$

$$= x0 \cdot y0 + x1 \cdot y1 + x2 \cdot y2 + x3 \cdot y3 + x4 \cdot y4 + x5 \cdot y5 + x6 \cdot y6 + x7 \cdot y7$$

The design can employ the FP multiplier and FP adder as IPs. As the design structure depicted in Figure 10.15, the *ddot* design's streaming width is set to one, permitting the utilization of a single FP multiplier for the multiplication-addition procedure. The accumulation of products generated in each multiplication iteration requires the utilization of multiple following FP adders. Consider

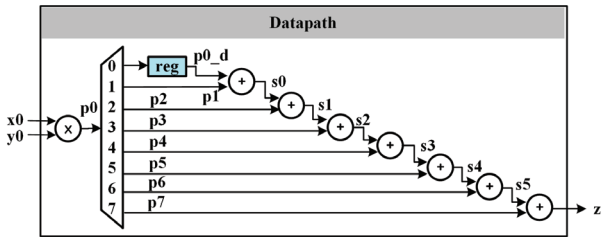

FIGURE 10.15
Iterative design datapath with streaming width one.

the significant demand for timing controllers to facilitate data processing efficiently in both a pipeline and parallel manner.

2) Create a testbench to facilitate the simulation of the *ddot* circuit. Validate the functionality of the circuit using the following specific test case: the FP vector x is composed of eight instances of FP 1.0, represented in IEEE-754 format as 32'h3f800000, and the FP vector y is comprised of eight instances of FP 2.0, represented in IEEE-754 format as 32'h40000000.

$$\begin{pmatrix} x0 \\ x1 \\ x2 \\ x3 \\ x4 \\ x5 \\ x6 \\ x7 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \\ 1.0 \end{pmatrix} \tag{10.3}$$

and

$$\begin{pmatrix} y0 \\ y1 \\ y2 \\ y3 \\ y4 \\ y5 \\ y6 \\ y7 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 2.0 \\ 2.0 \\ 2.0 \\ 2.0 \\ 2.0 \\ 2.0 \\ 2.0 \end{pmatrix} \tag{10.4}$$

T&F--Not for Distribution

PBL 29: Streaming Design on DFT2

1) Design a hardware circuit for the discrete Fourier transform of size 2 ($DFT2$), utilizing the signal-clock-cycle FP operators. The general formula for a DFT is a linear transform represented as:

$$y[k] = \sum_{n=0}^{N-1} x[n]\omega_N^{kn} \tag{10.5}$$

Here, x and y represent one-dimensional input/output arrays of size N. The symbol $\omega_N = e^{-j\frac{2\pi}{N}}$, where $j^2 = -1$. For a specific DFT size of 2, the complex output y can be represented as follows:

$$\begin{aligned} y[0] &= x[0]\omega_2^0 + x[1]\omega_2^0 = x[0] + x[1] \\ y[1] &= x[0]\omega_2^0 + x[1]\omega_2^1 = x[0] - x[1] \end{aligned} \tag{10.6}$$

From a hardware perspective, the complex inputs $x[0]$ and $x[1]$ can be represented by the real parts "x0_re" and "x1_re", as well as the imaginary parts "x0_im" and "x1_im". Similarly, the complex outputs $y[0]$ and $y[1]$ can be represented by the real parts "y0_re" and "y1_re", and the imaginary parts "y0_im" and "y1_im". These inputs and outputs are illustrated in Figure 10.16(a). In addition to the data interfaces, the "ready-valid" bus protocol is employed to indicate when data on the input buses is ready for use and to validate output data on the output buses. The clock, input, and enable signals are also essential IOs of the sequential circuit. It is important to note that all the registers in the design are triggered by the rising clock edge and rising reset edge.

A specific simulation example is illustrated in the timing diagram in Figure 10.16(b). Four sets of data are input into the $DFT2$ core:

- First complex input set: $x[0]$ is $1.0+j\cdot 0.0$, and $x[1]$ is $2.0+j\cdot 0.0$;
- Second complex input set: $x[0]$ is $2.0 + j \cdot 1.0$, and $x[1]$ is $2.0 + j \cdot 4.0$;
- Third complex input set: $x[0]$ is $1.0+j\cdot 2.0$, and $x[1]$ is $3.0+j\cdot 4.0$;
- Fourth complex input set: $x[0]$ is $4.0 + j \cdot 3.0$, and $x[1]$ is $2.0 + j \cdot 1.0$.

After two clock cycles delay, the corresponding outputs are pushed out sequentially in a pipeline manner.

T&F – Not for Distribution

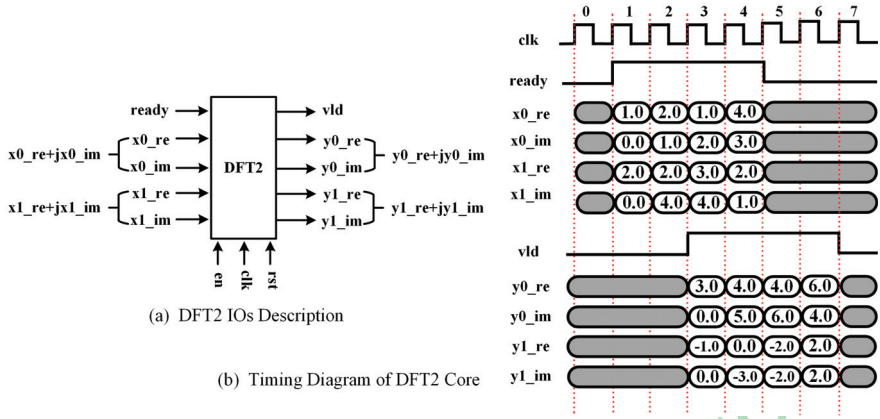

FIGURE 10.16
Design IOs and timing diagram of $DFT2$.

T&F – Not for Distribution

Design Hint: Following Equation 10.6, the $DFT2$ core can be implemented using the following FP operations:

$$\begin{aligned} y0_re &= x0_re + x1_re \\ y0_im &= x0_im + x1_im \\ y1_re &= x0_re - x1_re \\ y1_im &= x0_im - x1_im \end{aligned} \tag{10.7}$$

This design necessitates two FP additions and two FP subtractions, which can be constructed using the FP adders and FP subtractors as depicted in Figure 10.17.

It's worth noting that the ready-valid bus protocol can be devised with shift registers, aligning with the clock cycles required for the FP operational datapath. In addition, the multiplexers serve to diminish switching activities for input data to the FP operators, thereby reducing dynamic power consumption. Consequently, FP operations are triggered solely when the input data is ready for use on the bus. Finally, the last stage of registers is employed to register-out all the combinational outputs from the multiplexers.

2) Create a testbench to facilitate the simulation of the $DFT2$ core and validate its functionality using specific test cases shown in Figure 10.16(b). To clarify, the real part of complex input $x[0]$ can be driven on "x0_re" as 1.0, 2.0, 1.0, and 4.0 for the four input sets, and the imaginary part as "x0_im" with values 0.0, 1.0, 2.0, and 3.0. Similarly, "x1_re" and "x1_im" represent the real and imaginary parts of each complex number input for $x[1]$. It's important to note that they must be represented in IEEE-754 format.

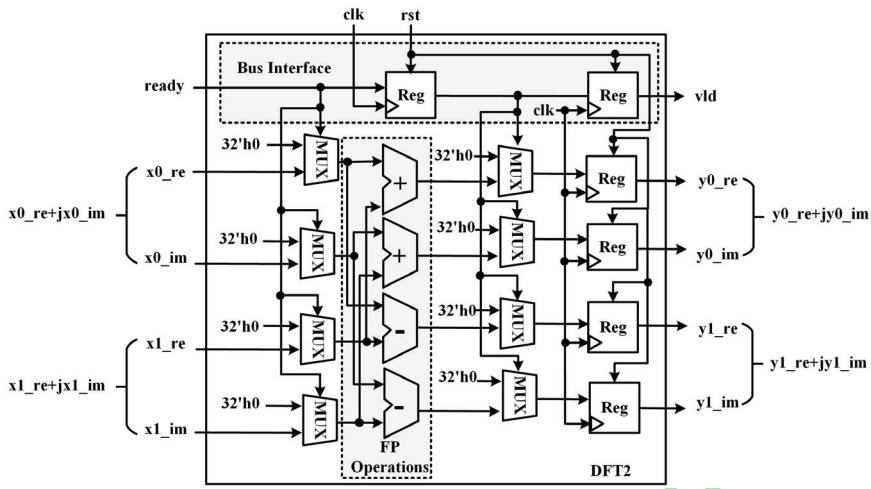

FIGURE 10.17
Block diagram of $DFT2$.

T&F—Not for Distribu

PBL 30: Streaming Design on DFT4

Design a hardware circuit for the discrete Fourier transform of size 4 ($DFT4$), utilizing the $DFT2$ core in PBL 29. Similar to PBL 29, the complex inputs and outputs, as well as the "ready-valid" buses are depicted in Figure 10.18(a). A specific simulation example is illustrated in the timing diagram in Figure 10.18(b). Two sets of data are input into the $DFT4$ core:

- First complex input set: $x[0]$ is $1.0+j\cdot 0.0$, $x[1]$ is $2.0+j\cdot 0.0$, $x[2]$ is $3.0+j\cdot 0.0$, $x[3]$ is $4.0+j\cdot 0.0$;
- Second complex input set: $x[0]$ is $-1.0+j\cdot 1.0$, $x[1]$ is $-2.0+j\cdot 2.0$, $x[2]$ is $-3.0+j\cdot 3.0$, $x[3]$ is $-4.0+j\cdot 4.0$.

After four clock cycles delay, the corresponding outputs are pushed out sequentially in a pipeline manner.

Design Hint: Following the Cooley-Tukey algorithm [6], you can create the $DFT4$ core by integrating four $DFT2$ cores, as illustrated in Figure 10.19. It's essential to incorporate the permutation operation into the $DFT4$ design. Below is the detailed guidance related to the permutation for $DFT4$ computation.

- For the interleaved interfacing, e.g., between **u2** and **u3**, the complex output $u2.y[0]$ from **u2** connects to the complex input $u3.x[1]$ from **u3**. Hence, the

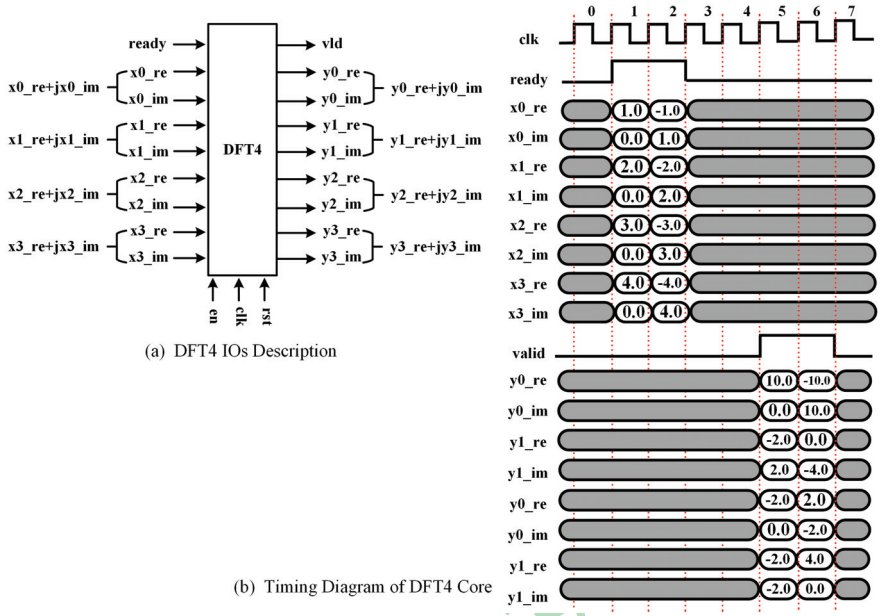

FIGURE 10.18
Design IOs and timing diagram of $DFT4$.

hardware connections should be $u3.x1_re = u2.y0_re$ and $u3.x1_im = u2.y0_im$.

- For the interfacing between **u2** and **u4**, the complex output $u2.y[1]$ from **u2** multiplies the imaginary unit $-j$, resulting in $-j \times (y1_re + j \cdot y1_im) = -j \cdot y1_re + y1_im$, and then feed it into the following complex input $u4.x[1]$ from **u4**. Hence, the hardware connections should be $u4.x1_re = u2.y1_im$ and $u4.x1_im = -u2.y1_re$.

2) Create a testbench to facilitate the simulation of the $DFT4$ core and validate its functionality using the specific test cases shown in Figure 10.18(b).

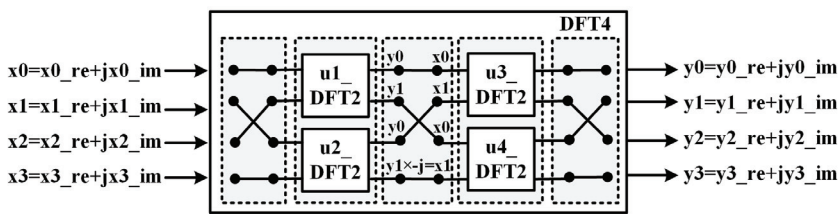

FIGURE 10.19
Block diagram of $DFT4$ using Cooley-Tukey algorithm.

T&F-Not for Dis